

Numerical Hypervolume and Magnitude in Dimensions Two to Ten

Sweeps in 2D and 3D; pairwise Chan recursion in 4D–10D

Research working report

4 October 2026

Abstract

We extend the numerical four-dimensional construction to every integer dimension from two to ten, including odd dimensions. The input is a finite set of nonnegative corners defining boxes anchored at the origin. The same coordinate-cell representation computes ordinary hypervolume and the dominated-set ℓ_1 magnitude by changing the one-dimensional masses. No symbolic integration or numerical quadrature is used.

Two and three dimensions use direct numerical sweeps with $O(n \log n)$ arithmetic complexity. In dimensions four through ten, a dimension-independent prefix-sum elimination kernel, paired coordinate order, weighted geometric cuts, and block-mass compression give $\tilde{O}_d(n^{d/3})$ arithmetic complexity. Odd dimensions finish each round with one singleton operation. Compression temporarily uses $2d$ variables, so the ten-dimensional implementation includes twenty-variable compression. An explicit local block schedule accounts for representation growth.

The accompanying standalone Python implementation supports all nine dimensions. Exact inclusion–exclusion tests, skyline-update tests, and compression tests accompany it. The exponent is Chan’s; this is a numerical formulation and implementation, with potentially large dimension-dependent constants, not a claim of improved exponents or competitive practical speed.

Contents

1	What is now available	3
2	The common numerical representation	4
3	Two dimensions: one descending sweep	4
4	Three dimensions: a numerical skyline tree	4
5	The common elimination kernel for dimensions four to ten	5
5.1	Comparisons remain numerical	6
5.2	Eliminating one variable	6
5.3	Pairs and the odd-dimensional final step	6
6	Compression by block masses in $2d$ variables	7
7	The geometric recursion in any fixed $d \geq 4$	7
7.1	Absorb one- and two-sided boxes	8
7.2	Weighted cuts and one full round	8
7.3	Integer weights and the block schedule	8
7.4	Explicit recursive procedure	9
8	Proof of the full arithmetic bound	9
9	Magnitude and hypervolume in both directions	10
10	Examples and use of the implementation	10
11	The Numba backend and measured performance	11
12	Validation and practical scope	12
13	Relation to Chan and the supplied repositories	13

1 What is now available

The implementation `numerical_chan.py` accepts dimensions $d = 2, 3, \dots, 10$. The earlier four-dimensional file remains a separate reference implementation. A user-facing call is

```
from numerical_chan import hypervolume, magnitude, NumericalChan
hv = hypervolume(points)                                # infer dimension
mag = magnitude(points)
solver = NumericalChan(dimension=7)
hv = solver.compute(points)
```

Here `points` contains nonnegative box corners; it is not a list of arbitrary lower and upper box endpoints. A reference-point conversion for conventional minimization data is given in Section 10.

Theorem 1 (Fixed-dimensional numerical bounds). *For every fixed $d \in \{2, \dots, 10\}$, the algorithm computes the hypervolume and dominated-set ℓ_1 magnitude of n anchored boxes. Its arithmetic complexity is*

$$\begin{cases} O(n \log(n+2)), & d = 2, 3, \\ O_d(n^{d/3} \log^{c_d}(n+2)), & 4 \leq d \leq 10, \end{cases}$$

where c_d is a constant depending only on d . All evaluations use finite numerical arrays, comparisons, integer searches, and arithmetic. No Cartesian product grid of coordinate cells is constructed by the solver.

Arithmetic has unit cost in this theorem. Exact rational arithmetic can have additional bit cost. Floating-point use can suffer cancellation; the theorem is not a numerical-error guarantee. The notation $\tilde{O}_d$ suppresses polylogarithms in n and constants depending on the fixed dimension. It does not suppress polynomial factors in n .

d	Numerical method	Bound	Variables in compression
2	Descending sweep	$O(n \log n)$	—
3	Sweep with a skyline tree	$O(n \log n)$	—
4	2 + 2 recursion	$\tilde{O}(n^{4/3})$	8
5	2 + 2 + 1 recursion	$\tilde{O}(n^{5/3})$	10
6	2 + 2 + 2 recursion	$\tilde{O}(n^2)$	12
7	2 + 2 + 2 + 1 recursion	$\tilde{O}(n^{7/3})$	14
8	2 + 2 + 2 + 2 recursion	$\tilde{O}(n^{8/3})$	16
9	2 + 2 + 2 + 2 + 1 recursion	$\tilde{O}(n^3)$	18
10	2 + 2 + 2 + 2 + 2 recursion	$\tilde{O}(n^{10/3})$	20

The pair notation specifies elimination and cut order. It does not assert that the geometric problem factors into independent two-dimensional problems. Relations between different pairs are retained.

2 The common numerical representation

For $P \subset \mathbb{R}_{\geq 0}^d$, let

$$D(P) = \bigcup_{p \in P} \prod_{i=1}^d [0, p_i].$$

We compute either $\text{HV}_d(P) = \lambda_d(D(P))$ or

$$\text{Mag}(D(P)) = \left(\bigotimes_{i=1}^d (\delta_0 + \tfrac{1}{2}\lambda) \right) (D(P)),$$

using Emmerich's dominated-set magnitude formula [2]. The measure includes mass on coordinate faces and at the origin.

On each coordinate axis, sort the distinct positive corner coordinates: $0 = c_{i,0} < c_{i,1} < \dots < c_{i,m_i}$. Use cells $\{0\}$ and $(c_{i,k-1}, c_{i,k}]$. Their numerical masses are

	$a_i(0)$	$a_i(k), k \geq 1$	
hypervolume	0	$c_{i,k} - c_{i,k-1}$	
magnitude	1	$(c_{i,k} - c_{i,k-1})/2$	(1)

Replace each corner by its rank vector $r(p)$. On cell indices k , box membership is exactly $k_i \leq r_i(p)$ for all i . Consequently, summing the product masses of covered index cells is exact. Only the d axis arrays are constructed, with total length at most $d(n+1)$; sorting and rank lookup cost $O_d(n \log n)$.

The anchor cell is retained even when it has zero hypervolume mass. The empty input has value zero for both measures, whereas a single origin point has hypervolume zero and magnitude one. Duplicate corners and tied coordinates require no general-position assumption.

3 Two dimensions: one descending sweep

Let $A_2(t) = \sum_{j=0}^t a_2(j)$, and set $A_2(-1) = 0$. Scan the first-axis index x in decreasing order. Before charging cell x , insert all boxes whose first coordinate rank is x . Maintain

$$H = \max\{A_2(r_2(p)) : r_1(p) \geq x\},$$

with empty maximum zero. Add $a_1(x)H$ to the answer. All intervals in the second coordinate share their anchor, so their union mass is the largest prefix mass. Each point is inserted once.

The scan itself is linear after sorting. Including preprocessing gives $O(n \log n)$ time and $O(n)$ space. This computes either measure simply by using the appropriate masses in (1).

4 Three dimensions: a numerical skyline tree

Again scan first-axis indices in decreasing order, inserting every box before charging its first-axis cell. The active cross-section is a union of anchored rectangles in coordinates

two and three. Let $A_3(t) = \sum_{j=0}^t a_3(j)$, with $A_3(-1) = 0$, and define

$$h(y) = \max\{A_3(r_3(p)) : p \text{ is active and } r_2(p) \geq y\},$$

where the empty maximum is zero. The cross-section mass is

$$V = \sum_y a_2(y)h(y). \quad (2)$$

The array h is nonincreasing. Inserting a rectangle with second rank u and third-coordinate prefix mass v performs the update

$$h(y) \leftarrow \max(h(y), v) \quad (y \leq u).$$

Lemma 2 (One interval changes). *Let ℓ be the first index with $h(\ell) < v$, or the array length if there is none. If $\ell > u$, nothing changes. Otherwise the update assigns the constant value v on exactly $[\ell, u]$.*

Proof. Nonincreasing order implies $h(y) \geq v$ before ℓ , and $h(y) < v$ at every subsequent index. The insertion affects only the prefix ending at u . Its changed set is therefore the stated interval. The updated array remains nonincreasing at both ends of that interval. $\square$

A lazy segment tree stores, for each index interval I , its minimum height and its weighted total $\sum_{y \in I} a_2(y)h(y)$. It also stores a pending constant assignment when one exists. The mass $\sum_{y \in I} a_2(y)$ is obtained from a fixed prefix array.

1. Find the first height below v by descending into the first tree child whose minimum is below v .
2. If it lies in $[0, u]$, assign v on $[\ell, u]$ with lazy propagation. An assigned node's weighted total becomes $v \sum_{y \in I} a_2(y)$.
3. Read V at the tree root, and add $a_1(x)V$ after all insertions at x .

Both tree operations cost $O(\log n)$, and there are n insertions and $O(n)$ first-axis cells. This proves the three-dimensional part of Theorem 1, with $O(n)$ space. Zero-weight cells cause no exception: comparisons concern heights, and a weighted sum can include zero masses.

5 The common elimination kernel for dimensions four to ten

This section gives the same finite-array kernel for any fixed number p of variables. During compression, p can be as large as $2d$. Each variable k_i ranges over a finite chain $E_i = \{0, \dots, s_i - 1\}$. A term is

$$T(k) = c \prod_i a_i(k_i) \prod_e \mathbf{1}[k_{j_e} \bowtie_e f_e(k_{i_e})], \quad i_e < j_e, \quad \bowtie_e \in \{\leq, \geq\}. \quad (3)$$

The unary arrays and scalar coefficient can be signed. Cutoff arrays are integer-valued and monotone. For each unordered pair retain at most four slots: upper or lower bound, each with nondecreasing or nonincreasing cutoff. Within one slot, combine upper bounds by minimum and lower bounds by maximum. Opposite monotonicities remain separate.

Thus a term uses $O_p(M)$ entries, where $M = \sum_i s_i$. Sentinels -1 and s_j encode empty upper and lower rays; no infinite numerical values or decorated open endpoints are needed.

5.1 Comparisons remain numerical

An active bound is either a constant or a monotone array $A(k_i)$. To impose $A(k_i) \leq B(k_j)$, use the following rules.

- If the two variables coincide, multiply their unary array by the Boolean comparison array. It may oscillate; unary weights need not be monotone.
- If one bound is constant, the comparison is also a unary Boolean array.
- For distinct variables, invert the monotone array on the larger-index variable by binary search. This produces one monotone cutoff in a pair slot.

Strict comparisons use the corresponding strict search boundary. The total work per comparison is $O_p(M \log(M + 2))$.

5.2 Eliminating one variable

For an index $q \in \{0, \dots, s - 1\}$ to eliminate, invert every incident pair predicate into a lower or upper bound on q . Include constant bounds 0 and $s - 1$. At most $2p - 1$ lower and $2p - 1$ upper candidates occur, because each other variable supplies at most two of each kind.

For each pair of candidate labels, impose the comparisons saying that L is the first maximizing lower bound and U the first minimizing upper bound, and impose $L \leq U$. Earlier labels are defeated strictly; later labels are defeated weakly. These regions are disjoint and exhaust the feasible assignments of the surviving variables.

Build the numerical prefix $P(t) = \sum_{q < t} a(q)$. On a label region, the eliminated sum is exactly

$$P(U + 1) - P(L). \quad (4)$$

Each summand is a scalar or a unary array in the variable supplying the bound. Multiply it into the surviving term with the appropriate sign. When both bounds use the same variable, combine their difference directly. The prefix need not be monotone: it becomes a unary weight, never an inverse cutoff.

Lemma 3 (Closure). *One application of SumOut emits at most $B_p = 2(2p - 1)^2$ terms in the same four-slot class, in $O_p(M \log(M + 2))$ arithmetic operations per input term.*

Proof. There are at most $(2p - 1)^2$ winning label pairs. Each emits at most two terms from (4). All label comparisons are of the forms just described; merging same-slot constraints preserves monotonicity. No new variable or new coordinate domain is introduced. A dimension-bounded number of array scans and binary searches gives the work bound. $\square$

5.3 Pairs and the odd-dimensional final step

For an ordered axis list $(1, \dots, d)$, apply SumOut to $(1, 2)$, then $(3, 4)$, and so on. If d is odd, eliminate d alone at the end. This is exactly the same one-variable operation, not a new odd-dimensional case or a padding dimension. Cross-pair constraints remain in the state. No Cartesian table on a coordinate pair is constructed.

6 Compression by block masses in $2d$ variables

Suppose a geometric cell has m surviving hard boxes. On each current axis, place a block boundary immediately after each surviving box cutoff strictly below the cell's upper endpoint. The blocks are contiguous integer intervals $I_i(t_i) = [L_i(t_i), R_i(t_i)]$, with at most $m + 1$ per axis. Define the desired coarse mass by

$$(\text{Push } F)(t) = \sum_{k_i \in I_i(t_i), 1 \leq i \leq d} F(k). \quad (5)$$

This equation specifies the answer; it does not instruct the implementation to enumerate the block products.

For each old term, add new variables $t_1, \dots, t_d$ with unit unary weights and the $2d$ membership comparisons $L_i(t_i) \leq k_i \leq R_i(t_i)$. There are now $2d$ variables. Eliminate only the old variables, in consecutive pairs with an odd final singleton when necessary. Rename the d surviving block variables.

Lemma 4 (Mass compression and reuse). *If the old state has S terms and total axis length M , compression emits at most $C_d S$ terms, where*

$$C_d = \prod_{p=d+1}^{2d} B_p.$$

It costs $O_d(S(M+m) \log(M+m+2))$ operations. The output uses $O_d(C_d S(m+1))$ array entries. For any future mask g constant on the current block products,

$$\sum_k F(k)g(k) = \sum_t (\text{Push } F)(t)g(t).$$

Proof. Apply the closure lemma d times. The combined old and new axis length is $M + O_d(m+1)$ and only decreases during elimination. Every remaining array is indexed by one coarse axis. For the final identity, partition the finite sum by block products and move the constant value of g outside each inner sum. Every surviving box cutoff was used to create the blocks; future box masks and cuts therefore have the required constancy. $\square$

No density is reconstructed and no division by cell measure is performed. An anchor may be merged into a neighbouring block when future predicates do not distinguish it; its mass is then retained in the block sum. For $d = 10$ this operation genuinely lifts to twenty variables and removes ten. The implementation does not substitute a four-dimensional compressor.

7 The geometric recursion in any fixed $d \geq 4$

The recursion stores the current index cell $\Gamma = \prod_i [\ell_i, u_i]$, a numerical state F , and the list B of unabsorbed boxes. Its invariant is

$$U(F, B, \Gamma) = \sum_{k \in \Gamma} F(k) \prod_{p \in B} (1 - \mathbf{1}[k \leq p]). \quad (6)$$

Initially $F = \prod_i a_i$. Return total bounding-box mass minus U . After compression, F stores coarse masses; the same invariant holds.

7.1 Absorb one- and two-sided boxes

Discard a box if some $p_i < \ell_i$. Otherwise call axis i active if $p_i < u_i$. A box with no active axis covers the cell, so $U = 0$. A box with one active axis i is absorbed by zeroing the corresponding unary entries $k_i \leq p_i$ in every term.

For a group of boxes with active axes $i < j$, build the suffix maximum

$$f(k_i) = \max\{p_j : p_i \geq k_i\}, \quad \max \emptyset = -1.$$

Their common uncovered predicate is $k_j \geq f(k_i) + 1$, a nonincreasing lower cutoff. Absorb it into the state and discard the group's boxes. The suffix array is built for the group, not separately for every box. Absorption never increases the number of terms. Every retained hard box has at least three active axes.

7.2 Weighted cuts and one full round

Cycle through axes $1, \dots, d$. Relative to the current axis, their cyclic positions are $0, \dots, d-1$. Each triple J of active axes of each hard box contributes weight

$$\omega(J) = 2^{\sum_{j \in J} \text{pos}(j)/d}.$$

Let Φ be their sum. For fixed d , Φ is within constant factors of the number of hard boxes. On the current axis, take a weighted median of the cutoff coordinates of triples containing that axis. Split after that index c into $[\ell, c]$ and $[c+1, u]$. These cells are disjoint, including at atomic anchors.

Lemma 5 (Potential contraction). *After the cut and cyclic advance, each child has potential at most $2^{-3/d}\Phi$. If no triple uses the axis, skip the cut and advance; the same bound holds.*

Proof. At most half of the weight of triples using the cut axis survives in either child. A surviving such triple increases its position sum by $d-3$, so its weight changes by $2^{(d-3)/d}$; half of this factor is $2^{-3/d}$. Every other triple reduces its position sum by three and changes by exactly $2^{-3/d}$. Restriction introduces no new active triple. If there is no candidate, only the latter case is present. $\square$

It follows that a full d -axis round has at most 2^d descendants, each with potential at most $\Phi/8$. This explains the exponent $d/3$, but a total-work proof must also charge the numerical state.

7.3 Integer weights and the block schedule

With n_0 original points, set $A = (n_0 + 2)^2$. A triple's position sum s is an integer from 3 through $3d-6$. Use

$$W_s = \left\lceil (A^d 2^s)^{1/d} \right\rceil$$

as its integer median weight. Binary search computes this root using integer powers; no floating-point root is used. The approximation satisfies

$$2^{s/d} \leq W_s/A \leq (1 + 1/A)2^{s/d}.$$

Thus the contraction gains at most a factor $1 + 1/A$ per level. Over $O_d(\log n_0)$ levels the total distortion is bounded by a constant. The combinatorial weights use $O_d(\log n_0)$ bits.

At the start of each block of geometric levels, set

$$Q = \left\lceil \frac{\sum W_s}{A} \right\rceil, \quad q = \max \left\{ 1, \left\lfloor \frac{\log_2 Q}{4d} \right\rfloor \right\}. \quad (7)$$

Run q full rounds, or dq levels. At each surviving block leaf absorb newly easy boxes, compress using the remaining hard cutoffs, and start a new block using its own Q . Skips consume one level just as cuts do. For odd d , a round consists of pairs and the last singleton axis.

7.4 Explicit recursive procedure

```
Solve(state, boxes, cell, axis, levels-left):
  discard disjoint boxes; absorb all easy boxes
  if a box covers cell or state is zero: return 0
  if at most b hard boxes remain: return Base(state, boxes, cell)
  if levels-left is zero:
    state = Push(state, remaining hard-cutoff blocks)
    relabel boxes and cell; initialize dq levels by (7)
  choose a weighted median on axis
  if none: recurse on same cell with next axis and one fewer level
  else: recurse on both disjoint children and add their answers
```

The first block is also initialized by (7). For a fixed constant b (default two), the base case expands the product in (6) into 2^b signed rectangle sums. Evaluate each rectangle by paired SumOut calls, with an odd final singleton. This inclusion–exclusion is constant-size; it is not applied to the full input list.

8 Proof of the full arithmetic bound

Discretization, absorption, disjoint splitting, exact one-variable sums, and Lemma 4 prove correctness inductively. The base case is the finite inclusion–exclusion identity, so the returned value is exact.

For the work bound, let $N = \Phi$ and S be the state term count at a block entry. Apart from the root, the axis length is $O_d(N)$ because the preceding compression retained only surviving hard cutoffs. The root has $O_d(n)$ entries and can be charged at root size even if many boxes are absorbed immediately.

Put $\beta_d = 1 - 3/(4d)$. For sufficiently large N , (7) gives $q = (\log_2 N)/(4d) + O_d(1)$. There are $O_d(2^{dq}) = O_d(N^{1/4})$ nodes and leaves in the block. Each is charged $\tilde{O}_d(SN)$ work, including compression or an early numerical base case. Each surviving child block has potential $O_d(N^{\beta_d})$ and at most $C_d S$ terms. Thus

$$T_d(N, S) \leq c_1 N^{1/4} T_d(c_2 N^{\beta_d}, C_d S) + \tilde{O}_d(SN^{5/4}). \quad (8)$$

All constants depend only on d . Once N is bounded by a fixed constant, only a bounded number of further levels is possible by geometric contraction, so the base cost is $O_d(S)$.

At macro-depth g , the largest size is at most $c_3 N^{\beta_d^g}$. There are $O_d(\log \log(N + 2))$ macro-generations, giving at most $C_d^g S = S \log^{O_d(1)}(N + 2)$ terms along a path. No assumption that empirical merging keeps the term count small is needed. The number of macro-nodes at depth g is at most

$$c_4^g N^{\frac{1}{4} \sum_{h < g} \beta_d^h} = c_4^g N^{\frac{d}{3}(1 - \beta_d^g)}.$$

Multiplying by the per-node charge gives, apart from polylogarithms,

$$S N^{\frac{d}{3}(1 - \beta_d^g) + \frac{5}{4} \beta_d^g} = S N^{\frac{d}{3} - (\frac{d}{3} - \frac{5}{4}) \beta_d^g} \leq S N^{d/3},$$

because $d/3 > 5/4$ for $d \geq 4$. Constants raised to g and the number of levels contribute only polylogarithmic factors. Summation proves $T_d(N, S) = \tilde{O}_d(S N^{d/3})$. At the root $S = 1$, $N = O_d(n)$; its possibly longer arrays cost at most $\tilde{O}_d(n^{5/4})$, also within the bound. Together with the sweeps this proves Theorem 1.

This particular block recurrence would not establish $\tilde{O}(n)$ for $d = 3$: its block-work exponent is $5/4$. That is why the implementation selects the skyline sweep there. Merely substituting $d = 3$ in the final formula would be unjustified.

9 Magnitude and hypervolume in both directions

For any dimension covered here, finite inclusion–exclusion and coordinatewise minima give

$$\text{Mag}(D(P)) = \text{HV}_d(\{\mathbf{1} + p/2 : p \in P\}). \quad (9)$$

Indeed, the mass of one anchored box is $\prod_i (1 + p_i/2)$, and the affine map commutes with intersections described by coordinatewise minima. The identity also holds for empty input.

Conversely, if every coordinate of every HV corner q is at least one,

$$\text{HV}_d(Q) = \text{Mag}(D(\{2(q - \mathbf{1}) : q \in Q\})).$$

For arbitrary strictly positive corners choose $s > 0$ with $s q_i \geq 1$; then

$$\text{HV}_d(Q) = s^{-d} \text{Mag}(D(\{2(s q - \mathbf{1}) : q \in Q\})).$$

Zero-coordinate boxes can first be discarded for this inverse HV transformation because their d -dimensional volume is zero. They must not be discarded for direct magnitude computation. If no boxes remain, return zero. These transformations preserve the polynomial exponent.

10 Examples and use of the implementation

The same two-box example works in every supported dimension:

$$p = (2, 1, 1, \dots, 1), \quad q = (1, 2, 1, \dots, 1).$$

Their intersection has every extent one. Hence

$$\text{HV}_d(\{p, q\}) = 3, \quad \text{Mag}(D(\{p, q\})) = \frac{15}{4} \left(\frac{3}{2}\right)^{d-2}.$$

An example call in seven dimensions is

```

points = [(2, 1, 1, 1, 1, 1, 1), (1, 2, 1, 1, 1, 1, 1)]
assert hypervolume(points) == 3
assert magnitude(points) == Fraction(3645, 128)

```

Here `Fraction` is imported from the standard-library `fractions` module. Integer and `Fraction` coordinates retain exact rational arithmetic. No third-party Python package is needed.

For minimization points a and a reference point r with $a_i \leq r_i$, convert to extents $p_i = r_i - a_i$. Then $\lambda_d(\cup_a[a, r]) = \text{HV}_d(\{r - a\})$ by reflection and translation. Input points outside this reference region must be handled explicitly; the core interface rejects negative extents. The magnitude interface applies to the resulting origin-anchored dominated set and its product measure, as defined above.

`NumericalChan(dimension=d)` fixes the expected dimension and checks every point. Without that argument, a nonempty input supplies its dimension; an empty input returns zero. All dimensions must lie between two and ten. Nonfinite or negative coordinates, mixed coordinate counts, and out-of-range dimensions are rejected.

The optional `block_levels` parameter is a test override. An arbitrary fixed override is not the schedule covered by the asymptotic theorem. The low-dimensional sweeps do not use it. Production calls should leave it unset. The implementation exposes counters through `solver.stats` for diagnostics.

11 The Numba backend and measured performance

The file `numerical_chan_numba.py` provides a separate `float64` backend. In dimensions two and three, the numerical sweep and skyline-tree operations run in Numba’s nopython mode. In dimensions four through ten, Numba compiles array scans, monotonicity checks, prefix sums, cutoff searches, normalization, and suffix masks. Geometric recursion, winning-bound enumeration, and structural term management remain Python. This is a hybrid implementation. No fastmath or parallel execution is used.

The exact standard-library backend remains available independently. The Numba version converts its coordinates and masses to floating point; it does not preserve exact rational arithmetic. A first call may include compilation or cache loading. Reported steady-state timings exclude that first call, and separate fresh-cache probes record first-use cost.

The reproducible benchmark compares the numerical Python and Numba backends with the repository’s unmodified Python implementations of Chan Section 4.2 ($d/3$) and Section 2 ($d/2$). The baseline commit is `08539aa99e94`; neither baseline uses Numba. All timed inputs are floating point. Dominance prefiltering is disabled, and input preprocessing and coordinate transforms are included. For magnitude, the references use the affine HV transformation.

Each entry below is the median of three calls after one untimed first call. Workers run sequentially. A 35-second limit covers the entire worker, including startup, its first call and all repeats; “timeout” is not a per-call runtime. Raw repeats and first-call times are in the accompanying JSON/CSV. No ratio is assigned to a timeout.

Case	Numerical Python	Numba	Chan 4.2	Chan 2
2D, $n = 256$	0.0014577	0.001104	—	0.013813
3D, $n = 128$	0.0017669	0.0006409	0.047406	0.015749
4D, $n = 24$	1.4637	2.3398	0.027249	0.0054424
4D, $n = 48$	timeout	timeout	0.10325	0.018492
5D, $n = 16$	1.1589	2.2592	0.049136	0.0066998
6D, $n = 12$	1.3261	2.8731	0.060131	0.0075414
7D, $n = 8$	0.75676	1.7606	0.055029	0.0070965
8D, $n = 10$	7.6582	timeout	0.19733	0.03268
9D, $n = 6$	timeout	timeout	0.03161	0.0032998
10D, $n = 8$	timeout	timeout	0.20869	0.023293

All times are seconds. The ND overview uses spherical, mutually nondominated fronts; the full data additionally includes tied-grid inputs and magnitude evaluations. These small cases do not measure asymptotic exponents.

For completed matched ordinary-HV cases in dimensions 4–10, the median ratio Numba numerical / Python numerical was 1.93x (20 cases). The median ratio Numba numerical / the repository’s Python Section 4.2 implementation was 21.25x (20 cases). Ratios above one mean the Numba version was slower. These are medians of per-case ratios for this small benchmark set, not universal performance predictions. Array-level JIT compilation does not remove the cost of enumerating and managing many terms. The results do not support a high-dimensional speedup claim for this implementation.

Across completed comparisons the maximum scaled disagreement was 3.31×10^{-16} , where scaling divides by $\max(1, |\text{reference value}|)$. This is an observed check, not a uniform error bound.

Fresh-cache first-use probes (module imports excluded) gave:

Dimension	First use (s)	Second use (s)
2	2.5352	9.72e-05
3	2.496	0.0001481
4	1.9391	0.0030841

The first call includes both compilation and computation; it is not pure compiler time.

Environment: CPython 3.12.14, NumPy 2.5.3, Numba 0.68.0, Windows.

12 Validation and practical scope

The verifier `verify_numerical_chan.py` uses independent exact oracles. In each of the nine dimensions it checks empty input, the origin, zero coordinates, duplicate and dominated corners, cyclic antichains, random positive integer instances, and rational coordinates against inclusion–exclusion. It also checks both directions of the affine magnitude–HV transformation.

The skyline tree is compared after each insertion with a dense one-dimensional array. Compression is checked at every lifted dimension $2d = 4, 6, \dots, 20$: the verifier explicitly enumerates small fine grids and compares their block sums with the emitted numerical

state. It then applies a new aligned cross-axis mask and compresses a second time. The Cartesian enumeration belongs only to the independent verifier.

For dimensions four through ten, additional instances force repeated geometric compression with a deliberately short test-only block length. These extend a genuinely four-dimensional instance by common extents in the additional coordinates, so the expected multiplicative factor is known. These structured tests exercise high-dimensional execution; they do not establish practical behaviour on large, fully coupled 10D inputs. Small general instances are tested separately.

The exact integer-root inequalities are checked directly. Geometric potential diagnostics use floating evaluation of the theoretical algebraic weights with a tolerance; measure comparisons and compression comparisons use exact arithmetic. Full counts and diagnostic states are recorded in `numerical_chan_results.json`. The exact suite passes 234 end-to-end cases, 248 first-compression cell checks, 18 second compressions, and 14 forced repeated-recursion instances. The separate Numba suite passes 144 end-to-end comparisons with an exact inclusion–exclusion oracle, 2,044 compression-cell comparisons, nine signed four-slot sums, six repeated-compression instances, and 320 direct checks of the compiled skyline update. Its maximum scaled end-to-end discrepancy in this suite is 3.60×10^{-14} . These are observed validation results, not a uniform error bound.

The constants C_d can be very large and the hidden logarithmic exponent depends on them. The prototype uses exact-key merging and normalization of unary factors to reduce practical state size, but the correctness and raw-emission proof do not require successful merging. If deterministic comparison-based term handling is desired, optional merging can be omitted or replaced by sorting. Even quadratic merging overhead in the polylogarithmic term count changes only a hidden logarithmic factor. These measurements do not establish an asymptotic speed advantage.

13 Relation to Chan and the supplied repositories

The geometric potential, absorption of locally one- and two-sided orthants, and periodic compression follow Chan’s orthant framework in Sections 4.1–4.2 of [1]. The simplification is its finite numerical realization: explicit cell masses, four monotone slots per coordinate pair, prefix-sum elimination, and block-mass pushforward. The locally chosen blocks make representation growth part of the proof.

The supplied **FastHVChan** repository [3] and **HV4DMagnitude** repository [4], including its **HV468DMagnitude** reports, supplied the implementation context. The earlier four-dimensional report described six- and eight-dimensional extensions mathematically; the present implementation and tests extend the actual numerical engine through ten dimensions, including five, seven, and nine. Dedicated 2D and 3D sweeps complete the requested range.

The construction preserves Chan’s high-dimensional exponent. It neither proves a sub- $n^{4/3}$ four-dimensional bound nor requires the previously unproved global batching lemma for that stronger target.

References

- [1] T. M. Chan. *Klee’s Measure Problem Made Easy*. Proceedings of FOCS 2013, pp. 410–419. DOI: 10.1109/FOCS.2013.51. Author’s August 2013 version, Sections 4.1–4.2: https://tmc.web.engr.illinois.edu/easyklee8_13.pdf.
- [2] M. T. M. Emmerich. *The Magnitude of Dominated Sets: A Pareto Compliant Indicator Grounded in Metric Geometry*. arXiv:2604.18147, 2026. <https://arxiv.org/abs/2604.18147>.
- [3] M. Emmerich and collaborators. *FastHVChan*. Repository snapshot 08539aa99e94, inspected 4 October 2026. <https://github.com/emmerichmtm/FastHVChan>.
- [4] M. Emmerich and collaborators. *HV4DMagnitude*, including the pair-elimination reports and the HV468DMagnitude code and audits. Repository snapshot 0c668e765202, inspected 4 October 2026. <https://github.com/emmerichmtm/HV4DMagnitude>.